

Representación de problemas y búsqueda

Estados, grafos y estrategias no informadas

Curso de Inteligencia Artificial
Semana 2 · Clase 2

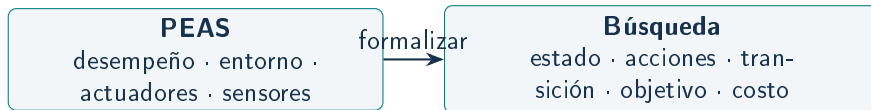
Propósito de la clase

Transformar una necesidad operativa en un **problema de búsqueda** y justificar una estrategia para resolverlo.

Al finalizar, podremos

- ▶ formular estados, acciones, transiciones, objetivo y costos;
- ▶ distinguir estado, nodo, camino, frontera y conjunto explorado;
- ▶ representar un espacio de estados como grafo;
- ▶ comparar estrategias no informadas y sus garantías;
- ▶ formular una búsqueda sobre el caso de movilidad.

De PEAS a una solución



Continuidad

PEAS define la tarea; la formulación de búsqueda define qué puede explorar el algoritmo.

Pregunta de apertura

Pregunta clave

¿Qué necesita conocer un algoritmo para encontrar una secuencia de decisiones?

- ▶ ¿Cómo describe cada situación posible?
- ▶ ¿Qué acciones son legales y qué cambia después de actuar?
- ▶ ¿Cómo reconoce una solución?
- ▶ ¿Qué significa que una solución sea mejor?
- ▶ ¿En qué orden debería explorar las alternativas?

Del agente al problema de búsqueda



La representación determina:

- ▶ qué soluciones existen y qué acciones parecen legales;
- ▶ qué costos se consideran;
- ▶ qué diferencias puede reconocer el algoritmo.

Una estrategia correcta no repara una representación incorrecta.

Formulación general

$$P = (S, A, T, s_0, G, c)$$

- ▶ S : espacio de estados.
- ▶ $A(s)$: acciones aplicables.
- ▶ $T(s, a)$: estado resultante.
- ▶ s_0 : estado inicial.
- ▶ $G(s)$: prueba de objetivo.
- ▶ $c(s, a, s')$: costo de transición.

Solución

Secuencia de acciones legales que lleva de s_0 a un estado que satisface G .

¿Qué es un estado?

Un **estado** reúne la información necesaria para:

- ▶ decidir qué acciones son legales;
- ▶ predecir consecuencias relevantes;
- ▶ evaluar si se alcanzó el objetivo;
- ▶ calcular costos futuros.

No es la realidad completa

Es una representación orientada a una decisión, un horizonte y un conjunto de restricciones.

Diseñar variables de estado

Movilidad

$s = (\text{zona}, \text{hora}, \text{vehículos disponibles}, \text{demanda pendiente})$

Zona

Hora

Disponibilidad

Demanda

Cada variable debe cambiar al menos una respuesta:

¿qué puedo hacer? · ¿qué ocurrirá? · ¿cuánto costará? · ¿alcancé el objetivo?

Estado, percepción y nodo

Concepto	Función
Estado del mundo	Situación que existe aunque no se observe por completo
Percepción	Evidencia recibida por sensores
Estado interno	Estimación mantenida por el agente
Nodo de búsqueda	Registro con estado, padre, acción, costo y profundidad

Dos nodos pueden contener el mismo estado y representar caminos distintos.

Suficiencia del estado

Un estado conserva la información del pasado que cambia el futuro relevante:

$$P(S_{t+1} \mid S_{0:t}, A_{0:t}) = P(S_{t+1} \mid S_t, A_t)$$

Ejemplo

Si una calle cierra a las 18:00, representar solo la zona no basta: la hora modifica acciones y transiciones.

Error frecuente

Omitir una variable para reducir el espacio y eliminar con ella una restricción decisiva.

Acciones, precondiciones y efectos

ACCIÓN trasladar

Precondiciones

- ▶ vehículo disponible;
- ▶ conexión habilitada;
- ▶ capacidad y seguridad válidas.

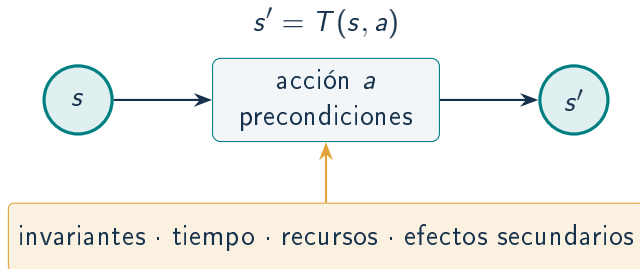
EFFECTOS

- ▶ cambia la disponibilidad;
- ▶ avanza el tiempo;
- ▶ aumenta el costo;
- ▶ cambia la cobertura.

Especificar solo el efecto favorable permite reutilizar recursos inexistentes.

Modelo de transición

En un problema determinista:



Los eventos externos cambian el estado, pero no deben atribuirse como acciones del agente.

Estado inicial y objetivo

Estado inicial s_0

Situación fechada desde la que comienza la búsqueda. Debe distinguir valores desconocidos.

Objetivo $G(s)$

Predicado que reconoce cualquier estado aceptable, no un único camino.

$G(s)$ = demanda crítica cubierta y restricciones satisfechas

El objetivo declara el resultado; no prescribe el método para alcanzarlo.

Longitud y costo

Para un camino de k acciones:

$$g = \sum_{i=0}^{k-1} c(s_i, a_i, s_{i+1})$$

Longitud

Cantidad de aristas o acciones: k .

Solo coinciden cuando todas las acciones cuestan una unidad.

Costo

Suma de los pesos de las transiciones.

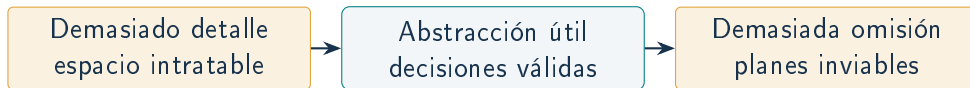
Costos y restricciones

Concepto	Tratamiento	Ejemplo
Restricción dura	Excluir alternativa	Capacidad o seguridad
Restricción blanda	Penalizar	Evitar reasignaciones
Costo	Comparar caminos	Minutos o kilómetros
Utilidad	Valorar consecuencias	Riesgo y calidad

Criterio normativo

Una ilegalidad no debe convertirse solamente en un costo alto: el optimizador podría aceptarla.

Nivel de abstracción



Una representación útil conserva diferencias que cambian:

- ▶ legalidad de las acciones;
- ▶ costo o consecuencia futura;
- ▶ cumplimiento del objetivo.

El estado correcto depende de la pregunta, el horizonte y las restricciones.

Ejemplo: una red vial

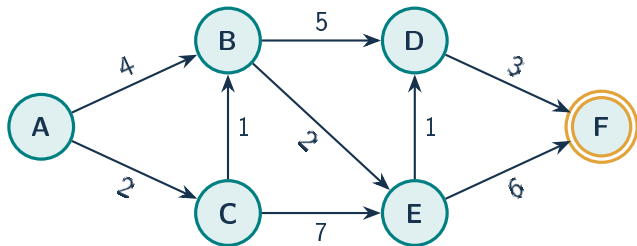
Objetivo: viajar de A a F minimizando minutos.

- ▶ **Estado:** intersección actual.
- ▶ **Acción:** recorrer una calle saliente.
- ▶ **Transición:** llegar al extremo de la calle.
- ▶ **Objetivo:** estado igual a F .
- ▶ **Costo:** tiempo indicado en la arista.

Prueba de suficiencia

Si existen cierres por horario o combustible limitado, la intersección deja de ser un estado suficiente.

Grafo dirigido y ponderado



Vértices = estados · aristas = transiciones · dirección = avance legal · peso = minutos

Grafo de estados y árbol de búsqueda

Grafo de estados

Contiene una vez cada estado y todas las transiciones legales.

$$A \rightarrow B \rightarrow D \quad \text{y} \quad A \rightarrow C \rightarrow D$$

Árbol de búsqueda

Contiene las distintas maneras de llegar: el estado D puede aparecer en dos nodos.

$$(A, B, D) \quad \text{y} \quad (A, C, D)$$

El dominio puede ser finito y el árbol de búsqueda infinito si contiene ciclos.

El nodo de búsqueda

$n = (\text{estado}, \text{padre}, \text{accion}, g, \text{profundidad})$

estado

padre

acción

g

profundidad

El nodo permite reconstruir el camino, comparar costos y conservar rutas alternativas.

Tres operaciones distintas

Generar un nodo · insertarlo en frontera · extraerlo y expandirlo.

La frontera

La **frontera** reúne nodos generados pero aún no expandidos.

Política de extracción	Estructura	Estrategia
Más antiguo	Cola FIFO	Anchura
Más reciente	Pila LIFO	Profundidad
Menor costo g	Cola de prioridad	Costo uniforme

Idea central

Cambiar la frontera cambia el orden de exploración, no la definición del problema.

El conjunto explorado

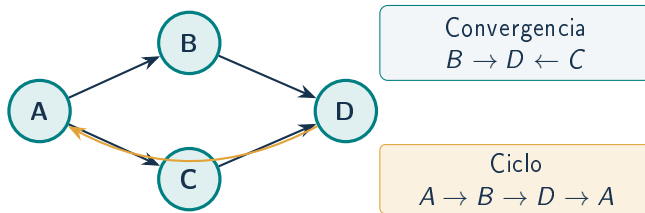
Registra estados ya expandidos para:

- ▶ evitar ciclos;
- ▶ reducir trabajo repetido;
- ▶ reconocer convergencias;
- ▶ conservar el mejor costo conocido cuando corresponde.

Error de implementación

Comparar objetos nodo por identidad no detecta estados equivalentes. Se necesita una clave canónica con significado en el dominio.

Ciclos y estados repetidos



Controles: ancestros · estados descubiertos · mejor costo y reapertura.

Esquema de búsqueda en grafo

```
insertar nodo inicial en frontera
mientras frontera no esté vacía:
    nodo <- EXTRAER(frontera)
    si ES-OBJETIVO(nodo.estado): devolver camino
    registrar nodo.estado como explorado
    para cada sucesor legal:
        nuevo_g <- nodo.g + costo_paso
        si es nuevo o mejora el costo:
            INSERTAR-O-ACTUALIZAR(sucesor)
devolver fracaso
```

La política de EXTRAER define la estrategia de búsqueda.

Tiempo y memoria

- ▶ b : factor de ramificación.
- ▶ d : profundidad de solución mínima.
- ▶ m : profundidad máxima.
- ▶ C^* : costo óptimo.
- ▶ ε : costo mínimo positivo.
- ▶ N : nodos generados o expandidos.

$$1 + b + b^2 + \dots + b^d = O(b^d)$$

Lectura

La mayoría de los nodos está en el último nivel; reducir el ramificado efectivo puede ser decisivo.

Búsqueda en anchura

BFS expande por niveles:

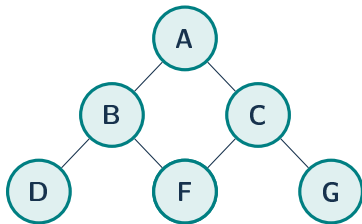
profundidad $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$

- ▶ Frontera: cola FIFO.
- ▶ Marca estados al descubrirlos.
- ▶ Encuentra primero una solución de menor profundidad.
- ▶ Conserva casi todo el nivel más reciente.

No confundir

Menor profundidad no implica menor costo si las aristas tienen pesos distintos.

BFS paso a paso



Extracción: A · B · C · D · E · F · G

- 1 desencolar el más antiguo;
- 2 comprobar objetivo;
- 3 encolar sucesores no descubiertos.

Garantías de BFS

BFS es:

- ▶ **completa** si b es finito y existe solución a profundidad finita;
- ▶ **óptima** cuando todas las acciones tienen igual costo;
- ▶ no óptima con pesos diferentes;
- ▶ costosa en memoria: tiempo y espacio $O(b^d)$.

Cuándo elegirla

Soluciones poco profundas, costos unitarios y memoria suficiente.

Búsqueda en profundidad

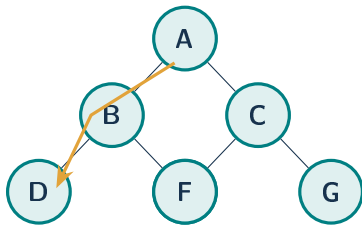
DFS expande el nodo más profundo disponible.

- ▶ Frontera: pila LIFO o recursión.
- ▶ Sigue una rama y luego retrocede.
- ▶ Usa menos memoria que BFS: $O(bm)$ en árbol.
- ▶ Puede quedar atrapada en ramas profundas o ciclos.

Garantías

No es completa en espacios infinitos y no garantiza el camino más corto ni el de menor costo.

DFS paso a paso



Un orden posible: A · B · D · E · C · F · G

El orden de sucesores debe fijarse para que la comparación sea reproducible.

Profundidad limitada

Impide expandir nodos más allá de un límite ℓ .

Solución

Objetivo encontrado.

Fracaso

No existe solución en el espacio revisado.

Corte

El límite impide concluir.

Selección del límite

Un máximo real de transbordos o pasos puede justificar ℓ ; un número arbitrario no.

Profundización iterativa

Ejecuta profundidad limitada con límites crecientes:

$$\ell = 0, 1, 2, 3, \dots$$

Combina:

- ▶ completitud y optimalidad por profundidad de BFS;
- ▶ memoria lineal de DFS;
- ▶ repetición controlada de niveles poco profundos.

Por qué la repetición es aceptable

En un árbol exponencial, el último nivel contiene la mayor parte del trabajo.

Búsqueda de costo uniforme

UCS extrae el nodo con menor costo acumulado g .

- ▶ Frontera: cola de prioridad.
- ▶ Expande por costo, no por profundidad.
- ▶ Conserva el menor costo conocido por estado.
- ▶ Actualiza o reabre si aparece un camino más barato.

Garantía

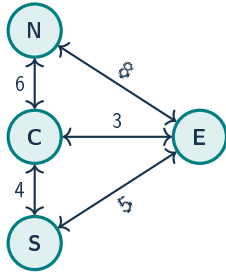
Es completa y óptima bajo costos no negativos y condiciones apropiadas de costo mínimo y ramificación.

Comparación de estrategias

Estrategia	Frontera	Completa	Óptima	Riesgo principal
BFS	FIFO	Sí*	Costos iguales	Memoria
DFS	LIFO	No general	No	Rama infinita
Limitada	LIFO + ℓ	Si $\ell \geq d^*$	No	Límite incorrecto
Iterativa	Límites crecientes	Sí*	Costos iguales	Repetición
UCS	Prioridad por g	Sí*	Sí*	Memoria y prioridad

* Las garantías solo valen bajo los supuestos declarados sobre el espacio y los costos.

Caso transversal: movilidad como grafo



- ▶ Estado: zona y configuración de flota.
- ▶ Acción: trasladar un vehículo.
- ▶ Transición: actualizar ubicación y tiempo.
- ▶ Objetivo: cubrir demanda crítica.
- ▶ Costo: demora + distancia vacía.

El grafo es una simplificación del sistema, no una copia de la ciudad.

Actividad guiada

En equipos, construir una instancia con 5 a 7 estados:

- 1 definir S, A, T, s_0, G, c ;
- 2 dibujar el grafo y asignar pesos;
- 3 ejecutar BFS y costo uniforme;
- 4 registrar orden de expansión, camino y costo;
- 5 explicar por qué los resultados coinciden o difieren;
- 6 identificar una variable omitida que cambiaría la solución.

Tiempo y entregable

30 minutos + 10 de contraste · tabla de formulación, grafo y comparación.

- ▶ Representar es decidir qué diferencias cambian acciones y costos.
- ▶ Estado y nodo no son sinónimos.
- ▶ La frontera define el orden de exploración.
- ▶ El conjunto explorado controla repetidos y ciclos.
- ▶ Longitud y costo solo coinciden con costos unitarios.
- ▶ Ninguna estrategia domina bajo todos los criterios.
- ▶ Las garantías dependen de supuestos explícitos.

Lecturas y continuidad

Lectura principal

- ▶ **Capítulo 6, sección 6.4:** representación computacional de problemas.
- ▶ **Capítulo 7, secciones 7.1 y 7.2:** espacios de estados y búsqueda no informada.

Próxima clase

Heurísticas · búsqueda voraz · algoritmo A* · admisibilidad · consistencia.

Representar bien precede a buscar bien.